

Kaggle TensorFlow Speech Recognition Challenge

Paweł Pozorski¹, Paweł Florek¹

¹ Warsaw University of Technology, Poland

{pawel.pozorski, pawel.florek}.stud@pw.edu.pl

Abstract

Keyword spotting (KWS) serves as the always-on front end of voice interfaces, where low latency and robustness are critical. We present a controlled experimental protocol and MLOps pipeline for comparing KWS architectures on the Kaggle TensorFlow Speech Recognition Challenge dataset. The study design isolates three factors: feature extraction strategy, architectural inductive bias, and non-command handling for `__unknown__` and `__silence__`. To ensure fair comparison, all models are trained from scratch under a centralized reproducibility framework with fixed seeds and deterministic controls. Final test evaluation is defined to operate only on Phase 4-selected systems, including single models and within-method ensembles across the three top Phase 3 backbones. This manuscript version documents the protocol and validation plan; empirical performance outcomes are intentionally deferred to a results-complete version.

Index Terms: keyword spotting, Kaggle TensorFlow Speech Recognition Challenge, MLOps, convolutional neural networks, transformers, state-space models, recurrent models

1. Introduction and Motivation

Detecting a fixed, limited vocabulary within an audio stream occupies a unique and highly resource-constrained niche in modern speech processing [1, 2]. Recent progress has diversified the model landscape, but many comparisons still confound architectural differences with inconsistent preprocessing and optimization choices. Our objective is to evaluate architectures under a common, from-scratch training protocol, ensuring that observed differences are attributable to structural inductive biases rather than transfer-learning advantages [3].

Our study is structured around three primary research questions, designed to systematically isolate the drivers of KWS efficacy:

- **Q1 (Feature strategy):** Which feature extraction pipeline produces the most generalizable representations for a fixed 1-second classification task? This evaluates whether the default reliance on standard log-mel spectrograms is genuinely optimal.
- **Q2 (Architectural inductive bias):** Which structural prior best captures the temporal-spectral dynamics of brief KWS inputs? We directly compare local spatial hierarchies, global all-to-all attention, and finite-memory sequential compression.
- **Q3 (Non-command class methodology):** Which methodology best recognizes `__unknown__` and `__silence__` while preserving macro-F1 on the core command classes?

To prevent hyperparameter explosion and to contextualize

model evaluation within a practical engineering framework, we structure our experiments into a strict four-phase pipeline (introduced in Section 3). In this manuscript version, the emphasis is methodological: the Results section specifies the held-out test protocol, while empirical comparisons are reported after all runs complete.

2. Dataset and Task Formulation

To provide a genuine evaluation of raw architectural capacity, this project leverages the Kaggle TensorFlow Speech Recognition Challenge dataset (derived from Google Speech Commands v1) [4] as an uncompromising test bed. These constraints eliminate substantial domain-level complexities that typically confound architectural comparisons.

While the canonical Speech Commands v0.01 release is designed to contain uniform one-second samples at 16 kHz, we observe that the Kaggle variant exhibits distinct deviations: approximately 89.8% of samples are exactly one second, but outliers range from 0.37 to 95.18 seconds. The full taxonomy contains 32 classes in total (30 command labels, plus `__unknown__` and `__silence__`). Models must therefore be formulated to robustly handle transient temporal padding while strictly discriminating across a multi-class landscape.

2.1. Unknown-Class Synthesis Methodology

For the extended dataset configuration, we generate additional `__unknown__` examples directly from the training corpus without external models. Candidate source clips are drawn from command classes only (excluding `__unknown__` and background-noise folders), with the hard constraint that clips within one synthetic sample must come from different classes.

Each synthetic item is formed by blending 2 or 3 one-second clips. Before mixing, each source is mean-centered, RMS-normalized toward a shared target level, and lightly smoothed with a short delayed component to reduce abrupt local transients. The final mixture is then produced by dense short-frame overlap where all sources remain active throughout the full second with slowly varying weights, encouraging spectro-temporal continuity rather than hard segment boundaries.

To avoid regenerating the same example, uniqueness is tracked at the source-file level (sorted source path tuple), not at the class-combination level. After synthesis, each candidate is passed through an envelope-based smoothness gate (windowed RMS trajectory checks for abrupt jumps and deep internal valleys). If a candidate fails this gate, it is discarded and regenerated in a retry loop; only accepted candidates are written to the `__unknown__` directory.

3. Experimental Design and MLOps Pipeline

To systematically address our research questions without combinatorial explosion, we filter configurations through a gated, four-phase funnel. Each phase selects the optimal setup based on macro-F1 across three fixed random seeds, locking that configuration for the next stage. Table 1 outlines the experiment scope and compute budget. Due to hardware limitations, all experiments in Phases 1–3 are executed on `train_small` and `valid_small` (Refer to Section 2 for exact definitions).

Table 1: *Estimated Model Training Calls per Experimental Phase.*

Phase	Experiment Scope	Configs	Seeds	Total Runs
1	Feature Strategy Ablation (5 features $\times$ 2 proxy models)	10	3	30
2	Global Hyperparameter Tuning (best feature $\times$ 2 proxies $\times$ 4 optim. combos)	8	3	24
3	Inductive Bias Comparison (AST: 8, ConvNeXt: 2, SSAMBA: 8, xLSTM: 8, MLP-Mixer: 4)	30	3	90
4	Non-Command Method Comparison (5 strategies $\times$ 3 top models on <code>__unknown__</code> / <code>__silence__</code>)	15	3	45
Total	Cumulative Training Executions	63		189

3.1. Phase 1: Feature Extraction Strategy (Q1)

Raw audio waveforms are highly inefficient for direct neural network processing due to a lack of semantic meaning at the sample level and high adjacent correlation [5]. We transform the signal into time-frequency representations using the Short-Time Fourier Transform (STFT) [6].

This extraction is governed by the time-frequency uncertainty principle (Gabor limit), bounding resolution such that $\Delta t \cdot \Delta f \geq \frac{1}{4\pi}$. We encapsulate all preprocessing pipelines within `PyTorch nn.Modules` to ensure differentiable, GPU-accelerated computations, evaluating five distinct strategies:

Mel-Spectrogram (Baseline): Emulating human auditory non-linearity via the Mel scale ($m = 2595 \log_{10}(1 + f/700)$) [7], this baseline uses $n_{fft} = 1024$ (64 ms window) and a 10 ms stride. Power spectra are filtered and compressed via an 80 dB-bounded log-amplitude transform to stabilize gradients.

High Temporal Resolution Mel: Shifting the Gabor limit temporally, this setup halves the window and hop sizes (32 ms window; 5 ms stride). The 200 Hz frame rate exposes sub-phonemic transients otherwise smeared by wider windows.

MFCC: Applying a Discrete Cosine Transform (DCT) to log-mel energies projects the signal into the quefrequency do-

main. Retaining 40 coefficients decorrelates channels [8] and slashes the data footprint by $\sim 93\%$.

PCEN: Per-Channel Energy Normalization [9] applies dynamic automatic gain control. It suppresses stationary noise and emphasizes transients [10], enhancing robustness against varied SNRs.

Mel + SpecAugment: Acting as structured input dropout, SpecAugment [11] applies stochastic rectangular masks across time and frequency axes, forcing the model to learn distributed phonetic representations.

To prevent favoring a specific architecture during this phase, we evaluate the pipelines on two proxy baselines: a spatial ConvNeXt model and a sequential xLSTM model. This ensures we do not inadvertently handicap sequential models that might uniquely benefit from high-temporal strides. Table 2 and Figure 1 detail these setups.

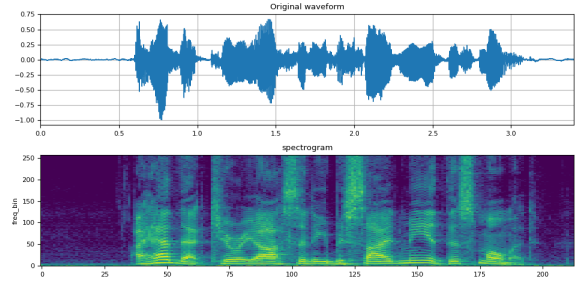


Figure 1: *Top: original waveform. After it, row by row: Mel-Spectrogram, High Temporal Resolution Mel, MFCC, PCEN, and Mel + SpecAugment.*

3.2. Phases 2 & 3: Architectural Bias and Optimization (Q2)

To isolate structural effects, we train all models entirely from random initialization. Rather than asking which pre-training source transfers best, this design asks which architecture inherently learns most effectively from the supervision signal.

Phase 2 locks the global optimization strategy (Table 3) using the optimal Phase 1 feature set. Phase 3 then sweeps model-specific hyperparameters (Table 4) across five distinct families:

AST (Audio Spectrogram Transformer): Replaces convolutions with self-attention [12, 3], granting a global receptive field from the first layer and assuming all-to-all interactions are necessary.

ConvNeXt (Modern CNN): Relies on translation equivariance and local correlation [13]. It modernizes residual networks with Vision Transformer elements, testing if local spatial hierarchies remain optimal [14].

SSAMBA (Selective State Space): Adapts Mamba [15] to audio [16]. By mapping 1D sequences via discretized differential equations, it achieves content-aware focus with RNN-like $O(L)$ efficiency.

xLSTM (High-Capacity Recurrence): Solves classical LSTM scalar bottlenecks by introducing matrix memory $C_t \in \mathbb{R}^{d \times d}$ [17], providing a pristine test of sequential tracking suitability.

MLP-Mixer: Serves as an architectural null hypothesis [18], applying alternating dense matrix multiplications without spatial or sequential priors.

Table 2: Feature extraction strategies compared in Phase 1.

Strategy	n_fft	hop_length	Window	Hop	Frames	Bins	Approx. input size (1 s)	Augmentation
Mel (baseline)	1024	160	64 ms	10 ms	101	128 mels	12,928 (128 × 101)	none
Mel (hi-temporal)	512	80	32 ms	5 ms	201	128 mels	25,728 (128 × 201)	none
MFCC	1024	160	64 ms	10 ms	101	40 MFCCs	4,040 (40 × 101)	none
PCEN	1024	160	64 ms	10 ms	101	128 mels	12,928 (128 × 101)	none
Mel + SpecAugment	1024	160	64 ms	10 ms	101	128 mels	12,928 (128 × 101)	freq+time masking

Table 3: Global optimization hyperparameters (Phase 2).

Parameter	Values	Purpose
Weight decay	$10^{-2}, 10^{-1}$	Prevents overwriting of learned features
LR scheduler	Cosine Annealing with Warmup / Reduce LR on Plateau	Cosine annealing with linear warmup; or metric-driven reduction

Table 4: Model-specific hyperparameter sweeps (Phase 3).

Model	Sweep Space
AST	Dropout {0.1, 0.5}; Head {linear, MLP(256)}; Pos. Emb. {interp., learned} [3]
ConvNeXt	Stoch. Depth {0.0, 0.2}; Kernel size {7 × 7} [14]
SSAMBA	Pooling {mean, max}; CLS {yes, no}; Stride {10ms, 5ms} [15, 16]
xLSTM	Dim $d \in \{32, 64\}$; State Reset {T, F}; Output {final, mean} [17]
MLP-Mixer	Dropout {0.0, 0.2}; Head L2 Norm {yes, no} [18]

3.3. Phase 4: Non-Command Class Methodology (Q3)

All core models are optimized natively via standard `CrossEntropyLoss` [19]. We intentionally defer imbalance-specific loss shaping and resampling in earlier phases to keep architectural comparisons interpretable. In Phase 4, we freeze the top three architectures from Phase 3 and vary only the non-command strategy, explicitly testing five alternatives for handling `__unknown__` and `__silence__`:

1. **Flat multiclass baseline:** unchanged single-head classification over all classes.
2. **Sampling control:** weighted random sampling with replacement and fixed target class priors: $p_{\text{unknown}} = 0.25$, $p_{\text{silence}} = 0.15$, and $p_{\text{cmd},k} = 0.02$ for each of the 30 command labels (summing to 1.0).
3. **Loss reweighting:** weighted cross-entropy with elevated penalties on non-command misclassification.
4. **Two-stage detector:** a separate binary gate (command vs. non-command), followed by command-word classification only when the gate predicts command.
5. **Shared two-head model:** a command head (30 labels) plus a dedicated non-command head (silence vs. unknown).

For sampling control, if class c has n_c training examples, each example from that class receives weight

$$w_c = \frac{p_c}{n_c}.$$

Each epoch draws N samples (with replacement), where N equals the original training-split size. This yields explicit undersampling of `__unknown__` whenever its empirical share exceeds 25%, and oversampling of `__silence__` and underrepresented command classes to match the target priors above.

To keep comparisons fair, all methods reuse identical data splits and three fixed seeds (0, 42, 2003). Model selection follows a strict gate: any method that degrades core-command macro-F1 by more than $\delta = 1.0$ percentage point relative to its frozen Phase 3 baseline is rejected. Among remaining candidates, we maximize non-command macro-F1,

$$\text{Macro-F1}_{\text{NC}} = \frac{\text{F1}_{\text{unknown}} + \text{F1}_{\text{silence}}}{2},$$

and use silence false-trigger rate and unknown-to-command leakage as tie-breakers.

The outputs of Phase 4 define the final deployable systems and are the only systems advanced to held-out test-set validation. We evaluate three inference flows: (i) single-stage multi-class flow (input $\rightarrow$ one model $\rightarrow$ class), (ii) two-stage detector flow (input $\rightarrow$ binary gate $\rightarrow$ either non-command label or command classifier), and (iii) shared-backbone two-head flow (input $\rightarrow$ shared encoder $\rightarrow$ command head and non-command head, with a fixed decision rule).

4. Strict Reproducibility and Training Control

To account for hardware-level non-determinism during GPU acceleration, we execute every experiment three separate times using fixed starting points (seeds 0, 42, and 2003) and report aggregated results (mean $\pm$ standard deviation).

All experiments are run on a single Apple MacBook Pro with an M4 Pro chip, 48 GB unified memory, and a 512 GB SSD. This hardware budget motivates the use of `train_small` and `valid_small` in Phases 1–3.

To reduce sampling bias introduced by subsampling, `train_small` and `valid_small` are constructed to preserve the label distribution $p(y)$ of their corresponding full splits (stratified downsampling).

We employ AdamW [20] to explicitly decouple weight decay from adaptive gradient scaling, and rely on Kaiming normal initialization [21]. To guarantee that experiments can be perfectly duplicated, we enforce deterministic data loading and automatically generate a `ReproducibilityReport` via MLflow [22] for every run. This report logs all environmental conditions, library versions, and the precise state of the codebase.

5. Results

Final test reporting is restricted to systems selected in Phase 4, since this stage determines deployment-relevant behavior for `__unknown__` and `__silence__`. After model selection on the validation split, each selected system is evaluated once on the

held-out test set. We report per-class precision, recall, and F1, together with command macro-F1 and non-command macro-F1.

Beyond single-model results, we evaluate all ensemble combinations across the three Phase 3-derived backbones within each fixed Phase 4 method. Denoting the model outputs by M_1, M_2, M_3 , we test all non-empty subsets (i.e., $2^3 - 1 = 7$ combinations) and aggregate class probabilities by simple averaging under the same within-method calibration rule. Cross-method ensembles are not considered; every ensemble is built from models produced by one common Phase 4 strategy. These ensemble checks are inference-only and are therefore excluded from the cumulative training execution counts in Table 1.

6. Conclusion

This work defines a reproducible, four-phase methodology for studying KWS under controlled conditions, from feature selection and architecture comparison to explicit non-command handling. The final systems are those selected in Phase 4, and the evaluation plan includes both single-model testing and within-method ensemble testing across the three best Phase 3 backbones. At this stage, the paper serves as a protocol and analysis blueprint rather than a source of empirical claims. Quantitative conclusions about architecture quality and boundary-class behavior will be drawn only after the full Phase 4 selection and held-out test evaluation are completed.

7. References

- [1] T. N. Sainath and C. Parada, “Convolutional neural networks for small-footprint keyword spotting,” in *Sixteenth annual conference of the international speech communication association*, 2015.
- [2] Y. Zhang, N. Suda, L. Lai, and V. Chandra, “Hello edge: Keyword spotting on microcontrollers,” *arXiv preprint arXiv:1711.07128*, 2017.
- [3] Y. Gong, Y.-A. Chung, and J. Glass, “Ast: Audio spectrogram transformer,” in *Interspeech 2021*, 2021, pp. 571–575.
- [4] P. Warden, “Speech commands: A dataset for limited-vocabulary speech recognition,” *arXiv preprint arXiv:1804.03209*, 2018.
- [5] H. Purwins, B. Li, T. Virtanen, J. Schlüter, S.-Y. Chang, and T. Sainath, “Deep learning for audio signal processing,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 13, no. 2, pp. 206–219, 2019.
- [6] J. B. Allen, “Short term spectral analysis, synthesis, and modification by discrete fourier transform,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 25, no. 3, pp. 235–238, 1977.
- [7] S. S. Stevens, J. Volkman, and E. B. Newman, “A scale for the measurement of the psychological magnitude pitch,” *The Journal of the Acoustical Society of America*, vol. 8, no. 3, pp. 185–190, 1937.
- [8] S. Davis and P. Mermelstein, “Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” *IEEE transactions on acoustics, speech, and signal processing*, vol. 28, no. 4, pp. 357–366, 1980.
- [9] Y. Wang, P. Getreuer, T. Hughes, R. F. Lyon, and R. A. Saurous, “Trainable frontend for robust far-field keyword spotting,” in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2017, pp. 5670–5674.
- [10] V. Lostanlen, J. Salamon, M. Cartwright, B. McFee, A. Farnsworth, S. Kelling, and J. P. Bello, “Per-channel energy normalization: Why and how,” *IEEE Signal Processing Letters*, vol. 26, no. 1, pp. 39–43, 2019.
- [11] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, “SpecAugment: A simple data augmentation method for automatic speech recognition,” in *Interspeech 2019*, 2019, pp. 2613–2617.
- [12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [13] Y. LeCun, Y. Bengio *et al.*, “Convolutional networks for images, speech, and time series,” *The handbook of brain theory and neural networks*, vol. 3361, no. 10, p. 1995, 1995.
- [14] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A convnet for the 2020s,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 11 976–11 986.
- [15] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.
- [16] S. Shams, S. S. Dindar, X. Jiang, and N. Mesgarani, “Ssamba: Self-supervised audio representation learning with mamba state space model,” *arXiv preprint arXiv:2405.11831*, 2024.
- [17] M. Beck, K. Pöppel, M. Spanring, A. Auer, O. Prudnikova, M. Kopp, G. Klambauer, J. Brandstetter, and S. Hochreiter, “xlstm: Extended long short-term memory,” *arXiv preprint arXiv:2405.04517*, 2024.
- [18] I. O. Tolstikhin, N. Houlsby, A. Kolesnikov, L. Beyer, X. Zhai, T. Unterthiner, J. Yung, A. Steiner, D. Keysers, J. Uszkoreit, M. Lucic, and A. Dosovitskiy, “Mlp-mixer: An all-mlp architecture for vision,” in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 24 261–24 272.
- [19] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: <http://www.deeplearningbook.org>
- [20] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.
- [21] K. He, X. Zhang, S. Ren, and J. Sun, “Delving deep into rectifiers: Surpassing human-level performance on imagenet classification,” in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 1026–1034.
- [22] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe *et al.*, “Accelerating the machine learning lifecycle with mlflow,” in *IEEE Data Eng. Bull.*, vol. 41, no. 4, 2018, pp. 39–45.